algorithms include Argon2, Bcrypt, and PBKDF2 in rough order of preference (see sidebar for details on secure password storage).

> **TIP** You may be able to avoid implementing password storage yourself by making use of an *LDAP* (Lightweight Directory Access Protocol) directory. LDAP servers often implement a range of secure password storage options. Another alternative is to make use of OAuth 2, which is covered in chapter 7.

Almost every week there is a story in the news about a hack or breach of a company's database, resulting in the theft of user details including stored passwords. The website haveibeenpwned.com lists several hundred hacked websites involving almost 8 billion user accounts, including many well-known companies such as LinkedIn and Adobe. Many users will reuse the same password for many different sites, making those passwords even more valuable to an attacker. After a database of passwords is compromised, attackers will then try the usernames and passwords from the database at many other sites to see if any of them match. This is known as *credential stuffing* and can be very effective at revealing passwords that have been reused elsewhere. So, it is critical that you securely protect your users' passwords even if you think your own API does not require high security.

Secure password storage

At first glance, you could just encrypt the passwords using a secret key known only to your application. Unfortunately, as you will see in later chapters, it is not easy to protect the key needed to decrypt those passwords, and so it may be compromised along with the